

C# PROGRAMLAMA DİLİNE GİRİŞ

Söz Dizim Kuralları

C# dili için oluşturulacak kaynak kod programcı tarafından metin dosyasına yazılırken belli **söz dizim kullarına** (**syntax**) uyar. C# dilinde de kaynak kodu oluşturan karakterler ön tanımlı olarak UTF-8 karakterleridir. Aşağıda belirtilen anahtar kelimelerle program yazılır.

Talimatlar ve Anahtar Kelimeler

Anahtar kelimeler (**keywords**), programlamada kullanılan ve C# derleyicisi tarafından özel anlama sahip, önceden tanımlanmış, **ayrılmış kelimelerdir** (**reserved word**). Başka hiçbir amaç için veya değişkenler veya yöntem veya sınıf adları gibi yerlerde kimliklendirme için kullanılamazlar.

C# sözdiziminin bir parçası olan önceden tanımlanmış sözcüklerdir ve **sınıf** (**class**), **alan** (**field**) ve **yöntem** (**method**) blokları içinde olan **talimatların** (**statement**), bir parçasıdır. **Talimatlar** yapısını gerekeni kısa ve net olarak belirten **mantıksal cümlelerdir** (**logical sequences**). C# programlama dilinde;

Anahtar kelimeler, küçük-büyük harf ayrımı yapıldığından her zaman küçük harflerle yazılırlar.

Yöntemler içinde yer alacak her talimat (**statement**), anahtar kelimeler içerecek şekilde yazılır ve noktalı virgül karakteri ile biter. Bu nedenle talimatlar **gelişigüzel** (**free format**) yazılabilir.

abstract	event	namespace	static
as	explicit	new	string
base	extern	null	struct
bool	false	object	switch
break	finally	operator	this
byte	fixed	out	throw
case	float	override	true
catch	for	params	try
char	foreach	private	typeof
checked	goto	protected	uint
class	if	public	ulong
const	implicit	readonly	unchecked
continue	in	ref	unsafe
decimal	int	return	ushort
default	interface	sbyte	using
delegate	internal	sealed	virtual
do	is	short	void
double	lock	sizeof	volatile
else	long	stackalloc	while
enum			

Tablo 6. Anahtar Kelime Listesi¹¹

C# dilinde anahtar kelime olmayıp **bağlam** (**context**) olarak kullanılan başka anahtar kelimeler de vardır;

add	field	not	select
allows	file	notnull	set
alias	from	nuint	unmanaged (function pointer calling convention)
and	get	on	unmanaged (generic type constraint)
ascending	global	or	value
args	group	orderby	var
async	init	partial (type)	when (filter condition)
await	into	partial (member)	where (generic type constraint)
by	join	record	where (query clause)

¹¹ <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/>

descending	let	remove	with
dynamic	nameof	required	yield
equals	nint	scoped	managed (function pointer calling convention)

Tablo 7. Bağlam Olarak Kullanılan Anahtar Kelime Listesi

Yüksek seviyeli bir dilde yazılmış bir **talimat** (**statement**), işlemciye belirtilen bir eylemi gerçekleştirmesini söyleyen cümledir. Yüksek seviyeli bir dildeki tek bir talimat, işlemciye gönderilen birden fazla CIL kodu ile gerçekleştirilir.

Açıklamalar

C# programlama dilinde tırnaklı parantez ile yazılan kodlar birbirinden ayrılır. Parantez açıldıktan sonra kapanana kadar yazılanlar **kod bloğu** (**code block**) olarak adlandırılır. İstenildiği kadar blok iç içe alınabilir Her **talimat** (**statement**), anahtar kelimeler içerecek şekilde yazılır ve noktalı virgül karakteri ile biter. Programcı kodu yazarken **açıklama** (**comment**) yapma gereği duyabilir. Bunu iki şekilde yapar;

1. Kod metninde bulunulan yerden sonra satırın sonuna kadar olan bir açıklama yapılmak istendiğinde, açıklama öncesinde iki adet bölü karakteri (//) kullanılır.
 2. Eğer birden çok satırı içeren bir açıklama yapılacak ise açıklama /* ile */ karakterleri arasına alınır.
- Aşağıda açıklama eklenmiş kod örneği bulunmaktadır.

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Merhaba!"); //Talimat ; ile biter.
/*
Bu program, açıklama (comment) içeren basit bir ana fonksiyonu olan C# programıdır.
İlhan ÖZKAN, Ocak 2025
*/
{ // Bi Kod Bloğu Başlangıcı
  { // İç Kod Bloğu Başlangıcı
    //iç bloğa ait girinti...
    Console.ReadLine(); //Konsoldan Satır Okuma Talimatı.
    //...
  } //İç Kod Bloğu Bitişi
} // Bir Kod Bloğu Bitişi
```

C# dilinde kodun bir başka programcı tarafından anlaşılması için bir kod bloğu içindeki açıklama ve talimatları bloktan itibaren girintili olarak yazarız. Kodu bu şekilde yazmamızın derleyici açısından hiçbir önemi yoktur. Derlemenin ilk aşamasında bütün bu açıklamalar ve beyaz boşluk karakterleri metinden çıkartılır.

Değişken Tanımlama

C# dilinde yöntemler, yapısal programlama dilindeki fonksiyonlar gibi tanımlanır. Yöntemlerde **veri yapıları** (**data structure**) ve **kontrol yapıları** (**control structure**) birbirinden ayırdır. Yöntem kodlanırken önce veriyi taşıyan veri yapıları tanımlanmalı daha sonra veriyi işleyecek kontrol yapıları tanımlanmalıdır.

Değişkenler veri yapılarının temel öğeleridir. **Değişkenin** (**variable**) ne olduğu *Değişken* başlığı altında verilmiştir. Değişkenler, belleğin belli bir bölgesinde yer kaplar ve çeşitli biçimde verilere ilişkin değerleri tutarlar.

Değer Tipler

C# dilinde bellekte yer işgal eden ve değerleri tutan değişkenler **value tipler** (**value type**) olarak adlandırılır:

1. **Tamsayı değer tipleri** (**integral value type**):

Takma Adı	Aralık	Bellek Boyutu	.NET Veri Tipi
sbyte	-128 ile 127	Signed 8-bit integer	System.SByte

byte	0 ile 255	Unsigned 8-bit integer	System.Byte
short	-32,768 ile 32,767	Signed 16-bit integer	System.Int16
ushort	0 ile 65,535	Unsigned 16-bit integer	System.UInt16
int	-2,147,483,648 ile 2,147,483,647	Signed 32-bit integer	System.Int32
uint	0 ile 4,294,967,295	Unsigned 32-bit integer	System.UInt32
long	-9,223,372,036,854,775,808 ile 9,223,372,036,854,775,807	Signed 64-bit integer	System.Int64
ulong	0 ile 18,446,744,073,709,551,615	Unsigned 64-bit integer	System.UInt64

Tablo 8. Tamsayı Değer Tipleri¹²

2. Kayan noktalı değer tipler (floating point value type):

Takma Adı	Yaklaşık Değer Aralığı	İncelik	Bellek Boyutu	.NET Veri Tipi
float	$\pm 1.5 \times 10^{-45}$ ile $\pm 3.4 \times 10^{38}$	~6-9 hane	4 byte	System.Single
double	$\pm 5.0 \times 10^{-324}$ ile $\pm 1.7 \times 10^{308}$	~15-17 hane	8 byte	System.Double
decimal	$\pm 1.0 \times 10^{-28}$ ile $\pm 7.9228 \times 10^{28}$	28-29 hane	16 byte	System.Decimal

Tablo 9. Kayan Noktalı Değer Tipleri¹³

- Mantıksal olarak **true** ya da **false** tutabilen **bool** değer tipi.
- Unicode UTF-16 karakterini tutan 16 bitlik **char** değer tipi. `'\u0000'` ile `'\uFFFF'` arasında değer alır. `'\u0000'` veya kısaca `'\0'` karakteri **null** karakter olarak adlandırılır.
- Yukarıda verilen değer tipleri içeren **yapılar** (**struct**) ve **numaralandırma** (**enum**) tipleri de değer tipleridir.
- Ayrıca **null** olabilir değer türü T? Yukarıda anlatılan değer tiplerinden olan T'nin tüm değerlerini ve ek bir **null** değeri temsil eder. Değer tipleri bu şekilde tanımlanmadığı sürece **null** atama yapılamaz.
- Değer **tuple** bir değer tipidir, ancak basit bir tip değildir.

void Tipi

Hiçbir değeri olmayan ve bellekte yer kaplamayan **void** veri tipi vardır. Bir yöntemin (veya yerel bir fonksiyonun) dönüş tipi olarak **void**, metodun bir değer döndürmediğini belirtmek için kullanırsınız. Ayrıca, bilinmeyen bir tipe gösterici **bildiriminde** (**declaration**) bir referans tipi **void** olarak kullanabilirsiniz.

Referans Tipler

Referans tiplerinin değişkenleri bellekte nesne referanslarını tutar. Değer tiplerinin değişkenleri doğrudan değerleri içerir. Referans tipleriyle, iki değişken aynı nesneye referans verebilir; bu nedenle, bir değişken üzerindeki işlemler diğer değişken tarafından referans verilen nesneyi etkileyebilir. Değer tiplerinde, her değişkenin kendi veri kopyası vardır ve bir değişken üzerindeki işlemlerin diğerini etkilemesi mümkün değildir.

Referans tipleri **bildirebilmek** (**declaration**) için **class**, **interface**, **delegate**, **record** anahtar kelimeleri ile nesneler oluşturulabilir. Bunların yanında C# dilinde **dynamic**, **object** ve **string** referans tipleri ön tanımlıdır.

Değişken Bildirimi

Değişkenler (**variable**) bellekte yer tahsis edilen depolama konumlarını temsil eder. Nesne yönelimli programlamada;

- Sınıflarda tanımlanan **alanlar** (**field**) değişkendir. Alanlar, sınıflardan imal edilen nesnenin durumlarına ilişkin verileri tutarlar.
- Yöntemler** (**method**) de yapısal programlamadaki fonksiyonlar gibi kodlanır. Yani yöntemler içinde ilk önce **veri yapıları** (**data structure**) tanımlanır daha sonra bu değişkenleri işeyecek kontrol

¹²<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/integral-numeric-types>¹³<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types>

yapıları tanımlanır. İşte yöntemler içerisinde ilk önce tanımlanacak veri yapılarının en basiti, değişkenlerdir. Bu değişkenler **yerel değişken** (**local variable**) olarak adlandırılır.

Her değişkenin, hangi değerleri depolanabileceğini belirten bir tipi vardır. Bu nedenle değişkenlerin veri tipine bağlı olarak kimlik verilerek **tanımlanması** (**definition**) gerekir. Yapılan bu tanımlama işlemine **bildirim** (**declaration**) adı da verilir. Bunun için;

1. Değişken tanımlaması yapılabilmesi için ilk önce **veri tipinin** (**data type**) belirlenmesi ve
2. Değişkenin kullanılacağı yerler göz önüne alınarak benzersiz bir **kimlik** (**identifier**) verilir.

```
// See https://aka.ms/new-console-template for more information
char yas;      // veri tipi "char" ve kimliği "yas" olan değişken bildirimi
int kat;       // veri tipi "int" ve kimliği "kat" olan değişken bildirimi
float kilo;    // veri tipi "float" ve kimliği "kilo" olan değişken bildirimi
decimal para; // veri tipi "decimal" ve kimliği "para" olan değişken bildirimi
```

Yukarıda tanımlaması yapılan **yas**, **kat**, **kilo** ve **para** değişkenleri ana fonksiyon olan **Program.cs** içinde tanımlandığından, tanımlanan değişkenler sadece program içinde geçerlidir. Tırnaklı parantez ile tanımlanan kod bloğu ya da program içinde benzer kimlik verilmiş bir başka değişkene izin verilemez.

Değişkenin kapsamı (**variable scope**); Bildirimi yapılan bir **değişkenin** (**variable declaration**), **erişilebileceği** (**access**), değişkenin üzerinde çalışılabileceği program kodu olarak tanımlanır. **Yerel değişkenler** (**local variable**) bulunduğu kod bloğu içerisinde bildirimi yapıp kullanılabilen değişkenlerdir.

Değişken Kimliklendirme Kuralları

C# programlama dilinde **değişkenlerin kimliklendirilmesinde** (**identifier definition**) aşağıda verilen kurallara uyulur;

1. Kimliklendirmede anahtar kelimeler kullanılamaz!
2. Kimliklendirme rakamla başlayamaz!
3. Kimliklendirmede yukarıdaki iki kurala uyacak şekilde UTF-8 kodlamasında herhangi bir karakter kullanılabilir.

```
// See https://aka.ms/new-console-template for more information
//Tamsayı Değişken Bildirimleri:
byte yaş;
int tamsayı;
long uzunTamsayı;
//Gerçek Sayı Değişken Bildirimleri:
float oran;
double gerceksayı;
/*Aynı veri tipinde birden fazla değişkene aralarına virgül koyarak kimlik verilebilir. */
int kat1, kat2, kat3;
float ölçek1, ölçek2;
decimal alınanÜcret, paraÜstü;
```

Bütün bu kuralların yanında değişkenlere kimlik verilirken yazılan kodun okunaklılarını artırmak için **Camel Case** olarak kimlik verilir. Bu kimliklendirme türünde ilk sözcük tamamıyla küçük, izleyen sözcüklerin ise baş harfi büyük yazılır. Bu kural zorunlu değildir ama koda bakım yapacak sonraki programcıların işini kolaylaştırmak için ahlaki olarak tercih edilir.

```
// See https://aka.ms/new-console-template for more information
int sayac;
int öğrenciBoy;
int asansörünOlduğuKat;
float asansörAğırlığı;
int öğrenciYaşı;
char ilkHarf;
float ortalamaNot;
```

Değişmezler

Programcı, kodlama yaparken bazı **değişmezleri** (**literal**) ister istemez kullanır. Bu değişmezler, derleme öncesinde de sonrasında da kaynak koddakinin kelimesi kelimesine aynıdır. Değişkenlere verilen **ilk değerler** (**initial value**) ile **katsayılar** (**coefficient**) en çok kullanılan değişmezlerdir. Değişkenlere ilk değer verme işlemine **başlatma** (**initialization**) adı verilir. Değişkenler tanımlanırken sahip olacağı ilk değerleri belirten değişmezler ile verilebilir;

```
// See https://aka.ms/new-console-template for more information
uint işaretSizTamsayı= 5U;
int tamsayı = -5;
sbyte işaretliBayt = -120;
decimal ondalık = 30.5M;
double çiftİncelikliKayanNoktalıSayı = 30.5D;
float kayanNoktalıSayı = 30.5F;
long uzunTamsayı = 5L;
ulong işaretSizUzunTamsayı = 5UL;
byte bayt = 127;
short kısaTamsayı = -12;
ushort işaretSizKısaTamsayı = 220;
bool evetHayır = true;
bool doğruYanlış = false;
char karakter1 = 'h'; // Tek tırnak içinde UTF-16 kodlamasına göre
char karakter2 = '\u006A';
char karakter3 = '\x0013';
char karakter4 = 'Ş';
/* Metin değişmezler çift tırnak arasında yazılır. Bu nedenle içinde çift tırnak kullanmak
   için \" olarak yazılır. Bir sonraki satıra geçmek için \ karakteri kullanılır. \
   karakteri kullanmak için \\ yazılır. */
string metin = "hello, this is a string literal";
/* Yukarıdaki metin değişmesinde birçok kural vardır. Onları gözardı etmek için @ karakteri
   ile başlayan * tam metin (verbatim string) değişmesi kullanılır. Sadece çift tırnak
   için iki adet "" kullanılır.*/
string tammetin= @"İlhan
    Özkan \ """;
```

Aşağıdaki kod örneğinde programcı, **3f**, **3.14f**, **3.14f** ve **2.0f** olmak üzere dört farklı değişmez kullanılmıştır. Bu değişmezlerin her biri derleyici tarafından farklı olarak değerlendirilir.

```
// See https://aka.ms/new-console-template for more information
float yariCap = 3f;
float daireninAlani = 3.14f * yariCap * yariCap;
float daireninCevresi = 2.0f * 3.14f * yariCap;
```

Yerel Sabitler

Değişmezlerin (**literal**) çokça kullanılması derleme sonrası üretilen icra edilebilir kodu da büyütür. Bunun yerine çok kullanılan değişmezler bellekte bir kez yer kaplasın diye **const** kelimesiyle **yerel sabitler** (**const**) tanımlanabilir. Bulunduğu kod bloğunda değişken gibi bildirim yapılmasına rağmen yerel sabitler değişken değildir ve değiştirilemezler. Sabit değişkenler etik olarak büyük harfle kimliklendirilir.

```
// See https://aka.ms/new-console-template for more information
const float PI=3.14f;
float yariCap = 3f;
float daireninAlani = PI * yariCap * yariCap;
float daireninCevresi = 2.0f * PI * yariCap;
```

Böylece aynı yerel sabiti **PI** birden çok yerde kullanılır ve her seferinde aynı bellek bölgesine erişildiğinden bellekten tasarruf edilmiş olunur. Bildirim sırasında const sıfatı kullanılan yerel sabite **ilk değer** (**initial value**), bir **değişmez** (**literal**) ile verilmelidir.

Bir sabit tanımlamanın üstünlükleri aşağıda sıralanmıştır;

- Gelişmiş Kod Okunabilirliği: Bir değişkene `const` sıfatı vermek, diğer programcılara değerinin değiştirilmemesi gerektiğini belirtir; bu da kodun anlaşılmasını ve sürdürülmesini kolaylaştırır.
- Gelişmiş Veri Tipi Güvenliği: `const` kullanılması, değerlerin yanlışlıkla değiştirilmemesini sağlayabilir ve kodumuzda hata ve eksiklik olma olasılığını azaltabilir.
- Gelişmiş Optimizasyon: Derleyiciler, değerlerinin program yürütme sırasında değişmeyeceğini bildikleri için `const` değişkenlerini daha etkili bir şekilde optimize edebilirler. Bu, daha hızlı ve daha verimli kodla sonuçlanabilir.
- Daha İyi Bellek Kullanımı: Değişkenlere `const` sıfatı vermek, değerlerinin bir kopyasını oluşturmayı engeller; bu da bellek kullanımını azaltabilir ve performansı artırabilir.
- Gelişmiş Uyumluluk: Değişkenlere `const` sıfatı vermek, kodunuzu `const` değişkenleri kullanan diğer kitaplıklar ve API'lerle daha uyumlu hale getirebilir.
- Gelişmiş Güvenilirlik: `const` kullanarak, değerlerin beklenmedik şekilde değiştirilmemesini sağlayarak kodunuzu daha güvenilir hale getirebilir ve kodunuzdaki hata ve eksiklik riskini azaltabilirsiniz.

Anonim Değişkenler

Değişkenler veri tipi verilmeden de tanımlanır. Bu durumda veri tipi değişkene verilen ilk değerden çıkarılır. Bu tip değişkenlere **anonim tip** (**anonymous type**) adı verilir. Anonim tipler **var** kelimesiyle tanımlanır ve statik tip denetimini korur yani tip denetimi derleme zamanı yapılır.

```
// See https://aka.ms/new-console-template for more information
var pi=3.14f;
var sayac=0;
var durum=false;
var gerçek=10.5;
var ondalıklı=100000.0m;

var anon = new { Value = 1 };
Console.WriteLine(anon.Value); //1
//Console.WriteLine(anon.Id); Derleme Hatası Olur!

var anon2 = new { x = 1, y = 2 };
// anon.x == 1
// anon.y == 2

int a = 10;
int b = 20;
var anon3 = new { a, b };
// anon3.x == 10
// anon3.y == 20

var anon4 = new { adı="ilhan", yaşı=50 };
// anon4.adı == "ilhan"
// anon4.yaşı == 50
```

Anonim tiplerin eşitliği de sorgulanabilir;

```
var anon = new { Foo = 1, Bar = 2 };
var anon2 = new { Foo = 1, Bar = 2 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon3 = new { Foo = 5, Bar = 10 };
var anon4 = new { Bar = 2, Foo = 1 };
// anon.Equals(anon2) == true
// anon.Equals(anon3) == false
// anon.Equals(anon4) == false (anon ve anon4 farklı tiplerdir)
```

Anonim tiplerin içerdiği özellikler aynı ise aynı olarak kabul edilir;

```
var anon = new { Foo = 1, Bar = 2 };
```

```
var anon2 = new { Foo = 7, Bar = 1 };
var anon3 = new { Bar = 1, Foo = 3 };
var anon4 = new { Fa = 1, Bar = 2 };
// anon ve anon2 aynı tiptir
// anon ve anon3 farklı tiptir (Bar ve Foo farklı sırada olduğundan)
// anon ve anon4 farklı tiplerdir (özellik isimleri farklı olduğundan)
```

Dinamik Değişkenler

Dinamik değişkenler, anonim değişkenlerin aksine, değişkenlerin tipi **dinamik** (**dynamic**) olarak derleme zamanı yerine çalıştırma zamanında veri tipi kontrolü yapılan değişken tiptir.

```
// See https://aka.ms/new-console-template for more information
dynamic val = "Merhaba";
Console.WriteLine(val.Id); // Derlenir, ama çalıştırma anında istisna olur!
```

Null Olabilen Değer Tipler

Değer tiplerin alabileceği alt ve üst sınırlar bellidir. Ancak bir değer tipe, değeri yok anlamında null atayamazsınız. Bunu sağlamak için değişkeni **null** olabilecek şekilde aşağıdaki şekillerde tanımlama yapılır;

```
int? i = null;
//Ya da
Nullable<int> i = null;
//Ya da
var i = (int?)null;
```

null olmayan bir değeri bu değişkenlere aşağıdaki şekilde yapılabilir.

```
int? i = 0;
//Ya da
Nullable<int> i = 0;
```

null olabilen bir veri tipini değer tipe aşağıdaki şekilde atayabiliriz.

```
int? i = 10;
int j = i ?? 0; // null ise 0 - null coalescing operator
int j = i.GetValueOrDefault(0); // null ise 0
int j = i.HasValue ? i.Value : 0; // null ise 0 - conditional tenary operator
```

Numaralandırma Tipi

Numaralandırma (**enumeration**) olarak adlandırılan **enum**, bir grup tamsayı ile ilişkilendirilmiş (integral) **değişmezden** (**literal**) oluşan numaralandırılmış kullanıcı tanımlı bir değer veri tipidir.

Bir **enum** aşağıdaki türlerden herhangi birinden türetilebilir: **byte**, **sbyte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**. Varsayılan olarak **int** veri tipi altta yatar ve **enum** tanımında veri tipi belirterek değiştirilebilir;

```
// See https://aka.ms/new-console-template for more information
byte işeBaşlananGün=1; // Pazartesi
Günler işbaşı = Günler.Pazartesi;
Aylar ramazan = Aylar.Mart;
Renkler arabanınRengi = Renkler.Kırmızı;

enum MedeniHal { Belirtilmemiş, Evli, Bekar, Boşanmış, Dul }; //Ön tanımlı olarak int
enum Günler : byte { Pazartesi = 1, Salı = 2, Çarşamba = 3, Perşembe = 4, Cuma = 5 }
enum Aylar : int { Ocak = 1, Şubat = 2, Mart = 3, Nisan = 4, Mayıs = 5, Haziran = 6,
                  Temmuz = 7, Ağustos = 8, Eylül = 9, Ekim = 10, Kasım = 11, Aralık = 12 }
enum Renkler : uint { Kırmızı = 1, Mavi = 2, Yeşil = 3, Sarı = 4, Beyaz = 5, Siyah = 6 }
```

Numaralandırmalar bir tamsayının her bir bitini temsil edecek şekilde **bayrak** (**flag**) olarak kullanılabilir;

```
// See https://aka.ms/new-console-template for more information
```



```
var İkiBayrakHavada = Bayraklar.BeşinciDurum | Bayraklar.ÜçüncüDurum;
Console.WriteLine(İkiBayrakHavada); //ÜçüncüDurum, BeşinciDurum
var ÜçüncüBayrakHavada = Bayraklar.ÜçüncüDurum;
Console.WriteLine(ÜçüncüBayrakHavada); //ÜçüncüDurum
var BaşkaİkiBayrakHavada = Bayraklar.BirinciDurum | Bayraklar.İkinciDurum;
Console.WriteLine(BaşkaİkiBayrakHavada); //BirinciDurum, İkinciDurum
```

```
[Flags]
enum Bayraklar
{
    //None = 0
    BirinciDurum = 1, //1.Bit
    İkinciDurum = 2, //2.Bit
    ÜçüncüDurum = 4, //3.Bit
    DördüncüDurum = 8, //4.Bit
    BeşinciDurum = 16 //5.Bit
}

[Flags]
enum FlagsEnum
{
    None = 0,
    Option1 = 1,
    Option2 = 2,
    Option3 = 4,
    Default = Option1 | Option3,
    All = Option1 | Option2 | Option3,
}
```

Yazdığımız Kod Nasıl İcra Edilir?

Derleyici tarafından işlemcinin anlayacağı makine diline çevrilen programımız aslında yazdığımız **talimatları** (**statement**) sırasıyla çalıştırır. C dilinde olduğu gibi her talimat noktalı virgül ile biter.

	Boy	Kilo	bki	boyKare
1 float boy = 1.80f;	1.80	?	?	?
2 float kilo = 100.0f;	1.80	100.0	?	?
3 float bki;	1.80	100.0	?	?
4 float boyunKare;	1.80	100.0	?	?
5 boyunKare = boy * boy;	1.80	100.0	?	3.24
6 bki = kilo / boyunKare;	1.80	100.0	30.86419753	3.24
7 boy = 1.50f;	1.50	100.0	30.86419753	3.24

Tablo 10. Örnek bir C# Programının İcra Sırası

Yapısal programlamada program, ana fonksiyondan başlar. Bu nedenle icra, **main** fonksiyonu içindeki ilk talimatla başlar ve solda verilen sırayla icra edilir. Sağda ise her icra sonrası değişkenlerin değerleri gösterilmiştir. Her talimatın icrasında neler olduğu aşağıda verilmiştir;

1. **boy** değişkenine **1.80** değeri atanır.
2. **kilo** değişkenine **100.0** atanır.
3. **bki** değişkeni tanımlanır.
4. **boyKare** değişkeni tanımlanır. Daha sonraki birkaç satır açıklama olduğundan icra edilecek bir şey yoktur.
5. **boy** ile **boy** değişkeni çarpılır ve sonuç **boyKare** değişkenine atanır. Artık **boyKare** değişkeninin değeri bu noktadan sonra **3.24** olmuştur.
6. **kilo** değişkeni **boyKare** değişkenine bölünür ve sonuç **bki** değişkenine atanır. Artık **bki** değişkeninin değeri bu noktadan sonra **30.86419753** olmuştur.
7. **boy** değişkenine **1.50** atanmıştır. Bu atama sonrası **boyKare** ve **bki** değişkenlerinin değeri değişmemiştir. Çünkü bu değişkenleri değiştiren 5. ve 6. talimatlar icra edilmiştir.

Temel İşleçler

İşleçler (operator) matematikten bildiğimiz işleçlerdir.

$$y = 3 + 2$$

Yukarıda verilen cebirsel ifadede toplama işleci (+) iki tarafına bulunan argümanları toplar. Bu argümanlara işlenen (operand) adı verilir. Yüksek düzey dillerde de aritmetik işleçler ve bunun yanında birçok işleç de bulunur. En çok kullanılan işleç, atama (=) işlecidir. Atama işleci, sağdaki işleneni soldakine atar. Bu nedenle kodlama yapılırken $2+2=x$ veya $a+b=x$ yazılamaz!

Aritmetik İşleçler

C# programlama dilinde aritmetik işleçlerin (arithmetic operator) çalışma şekli işlenenlerin (operand) veri tipine göre değişir.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
+	Sol ve sağındaki yer alan işlenenleri toplar.
-	Soldaki işlenenden sağdakini çıkarır.
*	Sol ve sağındaki yer alan işlenenleri çarpar.
/	Soldaki işleneni sağdakine böler. Eğer işlenenlerin her iki de tamsayı ise sadece kalan kısmı atılır. Bu durumda son uç yine tamsayı olarak üretilir.
%	Kalan (modulus) işleci, soldaki işlenenin sağdakine bölümünden kalanı verir.
++	Artırım (increment) işleci, işlenenin sağında veya solunda kullanılabilir. İşlenen solda ise işlenenin değeri kullanılmadan önce 1 artırılacaktır (pre-increment). İşlenen sağda ise işlenenin değeri kullanıldıktan sonra 1 artırılacaktır (post-increment).
--	Eksiltme (decrement) işleci, işlenenin sağında veya solunda kullanılabilir. İşlenen solda ise işlenenin değeri kullanılmadan önce 1 eksiltilecektir (pre-decrement). İşlenen sağda ise işlenenin değeri kullanıldıktan sonra 1 eksiltilecektir (post-decrement).
-	Tekli eksi: Kendisinden sonra gelen işlenenin işaretini değiştirir. İşlenen pozitif ise negatif yapar, negatif ise pozitif yapar.
+	Tekli +: Kendisinden sonra gelen işlenenin işaretini pozitif yapar.

Tablo 11. Aritmetik İşleçler

Artırım (++) ve eksiltme (--) işleçleri hariç olmak üzere bu işleçler `int`, `uint`, `long` ve `ulong` tiplerini işlenen olarak alır. İşlenenler diğer tamsayı tipler olduğunda (`sbyte`, `byte`, `short`, `ushort` veya `char`), değerleri bir işlemin sonuç tipi olan `int` tipine dönüştürülür.

```
// See https://aka.ms/new-console-template for more information
int a = 9, b = 4, res;
float fa = 6.6f, fb = 2.2f, fres;
res = a + b; // işlem sonucu res=13
res = a - b; // işlem sonucu res=5
res = a * b; // işlem sonucu res=36
res = a / b; // işlem sonucu res=9/4=2 tam 1/4=2
/* Toplama işlecinin işlenenleri tamsayı olduğundan, bölme sonucundaki tam kısım
   bölme sonucudur.*/
res = (int)(a / 4.3);
/* İşlem sonucu double olacaktır. Bu durumda res değişkeni de int olduğundan
   sonucu int türüne çevirmemiz gerekir. res=9/4.3=9.0/4.3=2.0930= 2 tam 0.0930=2 */
res = a % b; // işlem sonucu res=9%4= 2 tam 1/4=1
fres = fa / fb; // işlem sonucu res=6.6/2.2=3.00
```

C# programlama dilinde tamsayı bölme işlemi işlemcinin en basit yetenekleriyle çözülür;

```
// See https://aka.ms/new-console-template for more information
int a = 19 / 2; /* a = 2*9+1 = 9 */
int b = 18 / 2; /* b = 9 */
int c = 255 / 4; /* c = 4*63+3 = 63 */
int d = 45 / 4; /* d = 4*11+1 = 11 */
```

```
double aa = 19 / 2.0; /* aa = 9.5 */
double bb = 18.0 / 2; /* bb = 9.0 */
double cc = 255 / 2.0; /* cc = 127.5 */
double dd = 45.0 / 4; /* dd = 11.25 */
double ee = 45 / 4; /* ee = 11 çünkü bölme tamsayılar arasında yapılmıştır */
```

Mantıksal İşleçler

Tekli işleçler (**unary operator**) sadece bir işlenen üzerinde işlem yapar.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
!	Mantıksal DEĞİL işleci işlenenden önce kullanılır. İşleneninin mantıksal durumunu tersine çevirmek için kullanılır. İşlenen true ise false , false ise true verir.
&	Mantıksal VE: İşlenenlerin her ikisi mantıksal olarak true ise true sonucunu verir.
	Mantıksal VEYA: İşlenenlerin en az biri mantıksal olarak true ise true sonucunu verir.
^	Mantıksal ÖZEL VEYA: İşlenenlerin mantıksal olarak aynı ise false , değilse true sonucunu verir.
&&	Şartlı VE: Eğer iki işlenen true ise true verir. Eğer soldaki işlenen false ise sağdakine bakılmaz.
	Şartlı VEYA: Eğer iki işlenenden biri true ise true verir. Eğer soldaki işlenen true ise sağdakine bakılmaz.

Tablo 12. Mantıksal İşleçler

Aşağıda tekli işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 5, b = 6;
bool res;
res = (a>0) && (b<0); // işlem sonucu res=true
res = (a>0) || (b<0); // işlem sonucu res=true
res = !(a>0); // işlem sonucu res=false
```

Eşitlik ve İlişkisel İşleçler

İlişkisel işleçler (**relational operator**), işlenenlerin arasındaki ilişkiyi verirler. Bu işleçler, karşılaştırma doğruysa **true**, değilse **false** değerini döndürür.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
==	Eşit mi? İşleci: İki işlenenin değeri birbirine eşit ise true, değilse false verir
!=	Farklı mı? İşleci: İki işlenenin değeri birbirinden farklı ise true, değilse false verir
>	Büyük mü? İşleci: soldaki işlenenin değeri soldakinden fazla ise true, değilse false verir
<	Küçük mü? İşleci: soldaki işlenenin değeri soldakinden az ise true, değilse false verir
>=	Büyük veya eşit mi? İşleci: soldaki işlenenin değeri sağdakinden fazla ya da iki işlenen eşit ise true, değilse false verir
<=	Küçük veya eşit mi? İşleci: soldaki işlenenin değeri sağdakinden az ya da iki işlenen eşit ise true, değilse false verir

Tablo 13. İlişkisel İşleçler

Aşağıda verilen ilişkisel işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 5, b = 6;
bool res;
res = a < b; // işlem sonucu res=true
res = a <= b; // işlem sonucu res=true
res = a > b; // işlem sonucu res=0
res = a >= b; // işlem sonucu res=0
res = a == b; // işlem sonucu res=0
res = a != b; // işlem sonucu res=true
```

Bit Düzeyi İşleçler

Bit düzeyi işleçler (**bitwise operators**) özellikle ikilik tabandaki tamsayılarda karşılıklı bitler üzerinde yapılan işlemlerdir.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
&	Bit düzeyi VE: Tamsayının karşılıklı bitleri VE işlemine tabi tutulur. Sonuç tamsayıda karşılaştırılan iki bit 1 ise sonuç bit 1, değilse 0 olacak şekilde işlem yapar.
	Bit düzeyi VEYA: Tamsayının karşılıklı bitleri VEYA işlemine tabi tutulur. Sonuç tamsayıda karşılaştırılan iki bitin herhangi biri 1 ise sonuç bit 1, değilse 0 olacak şekilde işlem yapar.
^	Bit düzeyi ÖZEL VEYA: Tamsayının karşılıklı bitleri ÖZEL VEYA işlemine tabi tutulur. Sonuç tamsayıda karşılaştırılan iki bitin her ikisi farklı ise sonuç bit 1, değilse 0 olacak şekilde işlem yapar.
~	Bit düzeyi NOT: Tekli işleç olup sonrasında gelecek sayının bitlerini 1 ise 0, 0 ise 1 olacak şekilde işlem yapar.
<<	Bit düzeyi sola kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sola kaydırır. En anlamlı bit taşar. Taşan bu bit 1 ise işlemcinin durum kaydedicisinde taşma bayrağı 1 olur.
>>	Bit düzeyi sağa kaydırma: Tekli işleç olup sonrasında gelecek sayının bitlerini bir bit sağa kaydırır. En anlamsız bit taşar. Taşan bu bit 1 ise işlemcinin durum kaydedicisinde taşma bayrağı 1 olur.
>>>	Bit düzeyi sağa kaydırma işleci ile aynı işlemi yapar ancak ilk işlenen işaretli bir tamsayıdır.

Tablo 14. Bit Düzeyi İşleçler

Bu işleçler **int**, **uint**, **long** ve **ulong** tiplerini işlenen olarak alır. İşlenenler diğer tamsayı tipler olduğunda (**sbyte**, **byte**, **short**, **ushort** veya **char**), değerleri bir işlemin sonuç tipi olan **int** tipine dönüştürülür. Aşağıda tamsayılar üzerinde yapılan bit düzeyi işlemler gösterilmiştir;

```
// See https://aka.ms/new-console-template for more information
int a = 6; // 0110
int b = 2; // 0010
int bitwise_and = a & b; // 0010
int bitwise_or = a | b; // 0110
int bitwise_xor = a ^ b; // 0100
int bitwise_not = ~b; // 1101
int left_shift = b << 2; // 0100
int right_shift = b >> 1; // 0001
int c = -1;
int unsigned_right_shift = c >>> 3;
```

Atama işleçleri

Atama işleçleri (**assignment operator**) en çok kullanılan işleçlerdir. Atama işleçleri her zaman sağdaki değeri sola atar! Dolayısıyla atama yapılacak uygun **veri tipinde** (**data type**) bir **değişken** (**variable**) olmak zorundadır.

İşleç	İşlenenler (operand) üzerinde yapılan işlem
=	Sağdaki işleneni sola atar.
+=	Soldaki işleneni sağdaki işlenen üzerine toplar ve sonucu sola atar.
-=	Soldaki işlenenden sağdaki işlenenden çıkarır ve sonucu sola atar.
*=	Soldaki işleneni sağdaki işlenene çarpar ve sonucu sola atar.
/=	Soldaki işleneni sağdaki işlenene böler ve sonucu sola atar. Eğer işlenenler tamsayı ise kesirli kısım göz ardı edilir. Bu durumda sonuç tamsayıdır.
%=	Bu işleç tamsayılarla çalışır. Soldaki işleneni sağdaki işlenene böler ve kalanı sola atar.
<=>	İki işlenen tamsayının ikincisi kadar biti sola kaydırılır ve sonucu sola atar.

>>=	İki işlenen tamsayının ikincisi kadar biti sağa kaydırılır ve sonucu sola atar.
&=	İki işleneni mantıksal veya bit düzeyi VE işlemine tabi tutar ve sonucu sola atar.
=	İki işleneni mantıksal veya bit düzeyi VEYA işlemine tabi tutar ve sonucu sola atar.
^=	İki işleneni mantıksal veya bit düzeyi ÖZEL VEYA işlemine tabi tutar ve sonucu sola atar.

Tablo 15. Atama İşleçleri

Aşağıda verilen atama işleçlerin kullanımına ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 10; /* a değişkenine 10 değışmezi = işleciyle atanıyor.*/
a += 10; /* a=a+10; ile eşdeğer: a değişkenine 10 ekleniyor. */
a -= 10; /* a=a-10; ile eşdeğer: a değişkeninden 10 çıkarılıyor. */
a *= 10; /* a=a*10; ile eşdeğer: a değişkeni 10 kat yapılıyor.*/
a /= 10; /* a=a/10; ile eşdeğer: a değişkeni 1/10 kat yapılıyor.*/
a %= 10; /* a=a%10; ile eşdeğer: a değişkeninin 10 a kalanı bulunup kendisine atanıyor. */
int b = 12;
//b-5=a; // HATA: Atama sağdan sola yapıldığından böyle bir talimat yazılamaz!
```

İşleç Öncelikleri

Cebirsel ifadelerde nasıl ki önce çarpma ve bölme sonra toplama işlemleri yapılıyor. C programlama dilinde yazılan ifadelerde de **işleç öncelikleri** (operator precedence) vardır.

Aşağıdaki tabloda çok kullanılan C# işleçleri en yüksek öncelikten en düşüğe doğru listelenmiştir. Her satırdaki operatörlerin önceliği aynıdır.

İşleçler	Kategori
x.y, f(x), a[i], x?.y, x?[y], x++, x--, x!, new, typeof, checked, unchecked, default, nameof, delegate, sizeof, stackalloc, x->y	Temel
+x, -x, !x, ~x, ++x, --x, ^x, (T)x, await, &x, *x, true, false	Tekli
x..y	Aralık
switch, with	switch ve with ifadeleri
x * y, x / y, x % y	Çarpım
x + y, x - y	Toplam
x << y, x >> y, x >>> y	Kaydırma
x < y, x > y, x <= y, x >= y, is, as	İlişkisel ve tip testi
x == y, x != y	Eşitlik
x & y	Mantıksal ve Bit Düzeyi VE
x ^ y	Mantıksal ve Bit Düzeyi ÖZEL VEYA
x y	Mantıksal ve Bit Düzeyi VEYA
x && y	Şartlı VE
x y	Şartlı VEYA
x ?? y	null kaynaştırma
c ? t : f	Şart işleci
x = y, x += y, x -= y, x *= y, x /= y, x %= y, x &= y, x = y, x ^= y, x <<= y, x >>= y, x >>>= y, x ??= y, =>	Atama

Tablo 16. İşleçlerin İşlem Öncelikleri¹⁴

Aşağıda işleçlerin önceliklerine ilişkin örnek programı lütfen inceleyiniz.

```
// See https://aka.ms/new-console-template for more information
int a = 10, b = 5;
```

¹⁴<https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/#operator-precedence>

```

a = a - b + 2; // Öncelik Sırası +, -, = İşlem sonunda a=7
a = a / b + 2; // Öncelik Sırası /, +, = İşlem Sonunda a=7/5+2=3
a = 2 * 10 - 20 / 2 + 3 + (12 - 10); // Öncelik Sırası: (-) * / - + +
/*
İfadenin çözüm sırası:
> 2*10-20/2+3+2
> 20-20/2+3+2
> 20-10+3+2
> 10+3+2
> 13+2
> 15
*/

```

Kayan Noktalı Sayı Hesapları

Kayan noktalı sayıların IEEE-754 standardında ikili tabanda tutulduğu *Değişken* başlığında anlatılmıştı. Kayan noktalı sayılar olarak değişkenlere verdiğimiz onluk tabanda değerler aslında arka planda ikilik tabanda tutulur ve bu durum yazdığımız kodlarda garip davranışların ortaya çıkmasına sebep olur. Buna örnek olarak; hemen hemen her programcının yaptığı ilk hata, aşağıdaki kodun amaçlandığı gibi çalışacağını varsaymaktır;

```

float total = 0;
for(float a = 0; a != 2; a += 0.01f) {
    total += a;
}

```

Acemi programcı bunun 0, 0.01, 0.02, 0.03, gibi artarak, 1.97, 1.98, 1.99 aralığındaki her bir sayıyı toplayacağını ve 199 sonucunu, yani matematiksel olarak doğru cevabı vereceğini varsayar. Bu kodda doğru yürümeyen iki şey olur:

1. Yazıldığı haliyle program asla sonuçlanmaz. **a** asla 2'ye eşit olmaz ve döngü asla sonlanmaz.
2. Döngü mantığını **a < 2** şartını kontrol edecek şekilde yeniden yazarsak, döngü sonlanır, ancak toplam 199'dan farklı bir şey olur. IEEE754 uyumlu işlemlerde, genellikle bunun yerine yaklaşık 201'e ulaşır. Bunun olmasının nedeni, Kayan noktalı sayılarının onluk tabanda kendilerine atanan değerlerin, ikilik tabanda yaklaşık değerlerini temsil etmesidir. Bu duruma aşağıdaki klasik örnek verilir;

```

double a = 0.1;
double b = 0.2;
double c = 0.3;
if(a + b == c)
    Console.WriteLine ("Bu satır İcra Edilmez!"); //IEEE754 standardında işlem yaparsak.
else
    Console.WriteLine ("Kayan Noktalı Sayılar İkili Sayı Sisteminde"+
        " Tutulduğundan Yaklaşık Olarak Hesaplanır.");

```

Programcı olarak gördüğümüz şey onluk tabanda yazılmış üç sayı olsa da derleyicinin (ve altta yatan donanımın) gördüğü şey ikili sayılardır. 0.1, 0.2 ve 0.3, ona mükemmel bölmeyi gerektirdiğinden (ki bu onluk sayı sisteminde oldukça kolaydır), ancak ikili sayı sisteminde imkansızdır (bu sayılar, onluk sayı sistemindeki 1/3 sayısı gibi 0.33333333333333... şeklinde kesin olmayan biçimde depolanması gerektiğindendir);

```

/*64 bitlik kayan noktalı sayılar, tam sayı kısmı dahil olmak üzere
53 basamaklı hassasiyete sahiptir */
double a = 0.1;
//onluk sistemdeki 0.1 ikili sistemde kusurlu olarak saklanır
double b = 0.2;
//onluk sistemdeki 0.2 ikili sistemde kusurlu olarak saklanır
double c = 0.3;
//onluk sistemdeki 0.3 ikili sistemde kusurlu olarak saklanır
double f= a + b ;
//c değişkeni ile f nin onluk sistemde 0,3'e tam olarak eşit olmadığını unutmayın!

```

```
if (f==c)
    Console.WriteLine("Ok");
else
    Console.WriteLine("Garip Bir Durum Var"); // Konsola bu yazılır
```